leaner_cut — walkthrough & talking points

This is the "lean," procedural build of the 3-agent system: one file, no classes, everything runs locally against Ollama. Use this README to explain the code out loud while you're screen-sharing — every section below has a **spoken version** (in quotes, say it like you mean it) and a **why**, so you're never just reading code at the interviewer.

---

1. What this actually is, in one breath

> "Three agents — Extractor, Risk Analyst, Synthesizer — that can either run completely independently, or hand work to each other collaboratively. Every run is logged to SQLite, and I picked four different ways to wire them together, because the JD explicitly said the agents need to work independently *or* collaboratively depending on the task — so I wanted to actually have both, not fake one with the other."

The three agents themselves never change. What changes is **how they're wired together** — that's the whole design idea in this file. Four wiring patterns, one shared set of agents.

---

2. Agent flow — the four patterns

```
Pipeline (sequential): query -> [Extractor] -> advancements -> [Risk Analyst] -> risks -> [Synthesizer] -> report (each arrow is a direct function call, output passed as an argument) Parallel (independent): query -> [Extractor] \ query -> [Risk Analyst] |-- all three fire at once, same raw query, no cross-talk query -> [Synthesizer] / Supervisor (orchestrator-worker): query -> [Supervisor classifies] -> "extractor" -> [Extractor] only -> "pipeline" -> full Pipeline above Blackboard: query -> [Extractor] -> writes to DB [Risk Analyst] -> reads Extractor's row from DB -> writes to DB [Synthesizer] -> reads both rows from DB -> writes to DB (agents never call each other directly — they only ever read/write the shared table)
```

Every pattern ends the same way: whatever ran gets logged into the `runs` table, and `dashboard.py` renders that table regardless of which pattern produced it. That's deliberate — the persistence and presentation layer don't care how the agents were wired, only that they logged their work.

---

3. Section-by-section

`call_llm()` — the one place that talks to the model

```
def call_llm(system_prompt: str, user_prompt: str) -> str: res =
requests.post("http://localhost:11434/api/generate", json={...})
```

> "Every agent is really just a system prompt plus a call to this one function. I didn't want three different places in the code that know how to talk to Ollama — if I ever swap the model, or point this at a real API instead, there's exactly one function to change."

Why it looks like this: it's a raw `requests.post` to Ollama's local `/api/generate` endpoint, not a heavyweight SDK — because the only thing this function needs to do is send a system prompt and a user prompt and get text back. `timeout=600` is generous on purpose: this is CPU-only local inference on a 1.5B model, and a long synthesis prompt can genuinely take a while — I'd rather wait than have the demo throw a timeout mid-sentence.

`log_run()` / `get_latest_output()` — the shared blackboard

```
def log_run(agent_name, inputs, outputs): ... def get_latest_output(agent_name) -> str: ...
```

> "This SQLite table is the single source of truth for what every agent did. `log_run` is the write side — every agent calls it after it finishes, no exceptions. `get_latest_output` is the read side, and it's *only* used by the blackboard pattern — that's what makes blackboard actually different from pipeline: agents pull their input from this table instead of getting it handed to them directly."

Why the table is created inline (`CREATE TABLE IF NOT EXISTS`) instead of a `schema.sql` file: for a file this small, a separate schema file is one more thing to keep in sync and one more thing that can go missing. The table only has one shape, ever, so it lives right next to the function that writes to it.

`AGENT_PROMPTS` — one dictionary, not three functions

```
AGENT_PROMPTS = { "extractor": ("Extractor", "You are an expert research agent..."), "risk": ("Risk Analyst", "You are a technical risk analyst..."), "synthesizer": ("Synthesizer", "You are a principal technical editor..."), }
```

> "Every pattern below needs to look up 'what's Extractor's system prompt' or 'what's Extractor's display name' — so I made that one dictionary instead of three separate hardcoded prompts scattered across four functions. If I need to tune what the Risk Analyst does, I change it in exactly one place, and every pattern picks it up automatically."

This is also what makes `--agent extractor` (single-agent mode) and `--pattern parallel` share code cleanly — they both just index into this dict rather than duplicating the prompt text.

`run_single_agent()` — the "just run one" case

```
def run_single_agent(agent_key: str, query: str) -> str:
```

> "This is the direct answer to 'run it with just one of the agents' — no chaining, no other agent involved. Whatever text you give it goes straight to that one agent's system prompt, gets logged, done."

This is also what backs the `extractor` branch of the supervisor pattern — the supervisor doesn't reimplement "run one agent," it just calls this.

`run_pipeline()` — sequential handoff

```
def run_pipeline(query: str): advancements = call_llm(extractor_prompt, query) ... risks = call_llm(risk_prompt, f"Field: {query}\nAdvancements:\n{advancements}") ... report = call_llm(synth_prompt, f"...\n{advancements}\n\n...\n{risks}")
```

> "This is the classic version — Extractor's output literally becomes part of the Risk Analyst's prompt, and both of theirs become part of the Synthesizer's prompt. It's the simplest way to make three agents collaborate, and it's also the most fragile: if Extractor hallucinates something wrong in step one, that mistake is now baked into the input for steps two and three. I still start here because it's the easiest one to reason about out loud, and it's genuinely the right tool when a task truly has three dependent steps."

`run_parallel()` — independent execution

```
with ThreadPoolExecutor(max_workers=len(AGENT_PROMPTS)) as pool: results = list(pool.map(run_one, AGENT_PROMPTS))
```

> "This is the other half of the requirement — agents running *independently*. All three get the same raw query at the same time, none of them waits on or reads another's output. I used a thread pool instead of just calling them one after another, because if I'm claiming 'independent,' I want them actually running concurrently, not just independently-in-sequence — the wall-clock time should look different, not just the wiring."

Honest caveat, worth saying out loud if asked: on a single local Ollama instance, the model itself may still serialize requests under the hood — the concurrency is real at the code level (three threads, three in-flight HTTP calls), even if the GPU/CPU underneath processes them one at a time. That distinction — "my code is parallel, the backend may not be" — is exactly the kind of nuance that shows you're not just running code, you understand what it's actually doing.

`run_supervisor()` — orchestrator-worker

```
SUPERVISOR_PROMPT = "...decide: does it need the Risk Analyst and Synthesizer too..." def run_supervisor(query: str): decision = call_llm(SUPERVISOR_PROMPT, query)... if decision == "extractor": run_single_agent("extractor", query) else: run_pipeline(query)
```

> "This is the pattern I actually think is the most 'real' one, and it maps straight onto what I said in my UWM answer — a central controller decides which agent runs, instead of a human deciding in advance. Concretely: I make one cheap classification call first — 'does this query need risk analysis and a synthesized report, or is it just a factual lookup the Extractor can fully answer' — and only then do I call the agent(s) that decision actually requires. That's the difference between a workflow that always burns three LLM calls no matter what you ask, and one that reasons about what the task actually needs first."

Why this is the default (`--pattern` defaults to `supervisor`): it's the one pattern that self-selects between independent and collaborative depending on the task, which is *exactly* the literal requirement in the brief — "capable of working independently or communicating with one another ... depending on the tasks." The other three patterns are fixed shapes; this one is the shape that decides.

`run_blackboard()` — decoupled via shared state

```
def run_blackboard(query: str): advancements = call_llm(extractor_prompt, query) log_run("Extractor", query, advancements) advancements_from_board = get_latest_output("Extractor") # <- reads DB, not the variable above risks = call_llm(risk_prompt, f"...{advancements_from_board}")
```

> "This one looks almost identical to pipeline if you just read the output, but the mechanism underneath is different on purpose. In pipeline, Risk Analyst's input is the Python variable `advancements` — direct function call, direct handoff. In blackboard, Risk Analyst doesn't receive anything from the caller at all — it goes and reads the Extractor's row back out of the database itself. The agents are fully decoupled: I could kill the process after Extractor finishes, restart it, and Risk Analyst would still find its input sitting on the blackboard. That's the trade-off — more moving parts, but agents that don't need to know about each other's function signatures, only about the shared table."

Notice `advancements_from_board` is fetched *again*, deliberately, inside the Synthesizer's block too — not reused from the Risk Analyst's fetch. That's not a bug, it's the pattern: every agent reads the blackboard fresh, it never trusts a value handed to it in-memory by another step.

`PATTERNS` + `argparse` — the actual switch

```
PATTERNS = {"pipeline": run_pipeline, "parallel": run_parallel, "supervisor": run_supervisor, "blackboard": run_blackboard} ... if args.agent: run_single_agent(args.agent, args.query) else: PATTERNS[args.pattern](args.query)
```

> "This is the part that ties it together for a live demo. If you tell me to run just one agent, `--agent` skips all the pattern logic entirely and goes straight to `run_single_agent`. Otherwise, `--pattern` picks which of the four wiring styles handles the query. Same three agents underneath, every time — only the orchestration around them changes. That's the point I want to land: 'multi-agent design' is really a question of *how you wire the handoffs*, not how many different agents you write."

---

4. If asked "why four patterns, isn't that overkill for three small agents?"

> "Fair pushback. The three agents themselves stayed small on purpose — one prompt, one call, one log line each. What I spent the extra time on was the wiring, because that's actually where the interesting design decisions live, and where I could show I understand the trade-offs instead of just shipping the first thing that worked. In an actual live task, I'd default to whichever one pattern the task obviously calls for — usually pipeline or supervisor — and only reach for parallel or blackboard if the task specifically needed 'these must run independently' or 'these must not know about each other directly.'"

---

5. Running it

No API key, no `REHEARSAL_MODE`, no provider switch needed — this file only ever talks to your local Ollama model (`OLLAMA_MODEL`, default `qwen2.5:1.5b`). Every run appends to `blackboard.db`; run `python dashboard.py` afterward to see it rendered.

From inside `leaner_cut/`

```
cd leaner_cut # --help — see every flag python main.py --help # Default run — no args at all, uses the built-in sample query and the # default pattern (supervisor) python main.py # Each of the four multi-agent patterns, explicit topic python main.py --pattern pipeline "quantum computing" python main.py --pattern parallel "quantum computing" python main.py --pattern supervisor "quantum computing" # same as omitting --pattern python main.py --pattern blackboard "quantum computing" # Force exactly one agent, independently — bypasses pattern selection entirely python main.py --agent extractor "what color is a tomato" python main.py --agent risk "what color is a tomato" python main.py --agent synthesizer "what color is a tomato" # Swap the local model for a run, without touching code $env:OLLAMA_MODEL = "qwen2.5:3b" python main.py --pattern pipeline "quantum computing" Remove-Item Env:\OLLAMA_MODEL # back to the qwen2.5:1.5b default # Render the dashboard after any run above python dashboard.py
```

From the repo root

```
python leaner_cut\main.py --agent extractor "your topic" python leaner_cut\main.py --pattern blackboard "your topic" python leaner_cut\dashboard.py
```

Via the venv directly (no activation), from the repo root

Useful when you haven't run `.\venv\Scripts\Activate.ps1` yet in the current shell — this is exactly how every example in this README was actually tested:

```
.\venv\Scripts\python.exe leaner_cut\main.py --pattern parallel "quantum computing" .\venv\Scripts\python.exe leaner_cut\main.py --agent extractor "what color is a banana"
```

Bash / Git Bash equivalents

Same script, just POSIX path separators and `python3`:

```
cd leaner_cut python3 main.py --pattern supervisor "quantum computing" python3 main.py --agent extractor "what color is a tomato" python3 dashboard.py
```

Via `run_rehearsal.ps1` (repo root)

The wrapper script that also auto-generates the dashboard on success:

```
.\run_rehearsal.ps1 # defaults: leaner_cut, supervisor pattern is not wired here — runs full pipeline
.\run_rehearsal.ps1 -Query "your topic" .\run_rehearsal.ps1 -Query "your topic" -Agent extractor #
single-agent mode .\run_rehearsal.ps1 -Query "your topic" -Model qwen2.5:3b # different local model
.\run_rehearsal.ps1 -Query "your topic" -Project original_shape # run the OOP version instead
```

Note: `run_rehearsal.ps1` predates the `--pattern` flag and only knows about `--agent` vs the full pipeline — for `parallel`/`blackboard`, call `main.py` directly as shown above rather than through the script.

---

6. Full source, annotated line-by-line

This is the actual, current content of `main.py` and `requirements.txt`, with a comment on every line explaining what it does and — where it matters — what the alternative would have been and why it lost. This is the copy to have open if you want to narrate the code top-to-bottom without having to improvise a reason for every choice on the spot. Nothing here changes the real files; it's a reference copy.

`main.py`

```
import argparse # stdlib CLI parser — chosen over hand-rolled sys.argv # indexing so --agent/--pattern get
free validation, # --help text, and clear errors on bad input for free. import os # only used to read env
vars (OLLAMA_MODEL) — no need # for anything heavier like python-dotenv's os wrapper. import sqlite3 # stdlib
DB driver — no server process to run, no extra # dependency; a file-based DB is exactly right for a #
single-process demo that just needs to persist rows. from concurrent.futures import ThreadPoolExecutor # used
only by the parallel pattern, to fire all three # Ollama calls concurrently instead of one after another. #
Chose threads over asyncio/aiohttp: these calls are # blocking `requests.post`s, and threads parallelize #
blocking I/O without rewriting call_llm as async. from pathlib import Path # used for __file__-relative paths
— chosen over plain # string concatenation so path joins are OS-correct on # both Windows and POSIX without
extra code. import requests # simple synchronous HTTP client — chosen over the # heavier `httpx` or writing
raw `urllib` calls, since # this script only ever does a single POST per call. from dotenv import load_dotenv
# loads a .env file into os.environ — chosen over asking # the user to export env vars by hand every session.
# Load .env next to this file, not from the caller's cwd — load_dotenv() # with no path searches cwd upward,
which silently misses it if this # script is invoked from outside its own directory.
load_dotenv(Path(__file__).parent / ".env") # explicit path, not load_dotenv() with no args — the # no-args
version searches upward from the *current # working directory*, which breaks the moment this # script is run
from anywhere but its own folder. DB_PATH = str(Path(__file__).parent / "blackboard.db") # DB always lives
next to this script, not in # whatever directory you happened to run it from — # same reasoning as the .env
path above. # 1. Base LLM caller — always the local Ollama model (OLLAMA_MODEL, default # qwen2.5:1.5b). No
API key or network provider involved. def call_llm(system_prompt: str, user_prompt: str) -> str: # one shared
function every agent calls — # chosen over each agent making its own HTTP # call, so there's exactly one
place to change # if the model or endpoint ever changes. res = requests.post(
"http://localhost:11434/api/generate", # Ollama's local generation endpoint — hardcoded # since this is
always local, never a remote host; # a remote LLM would need auth/TLS this doesn't have. json={ "model":
os.environ.get("OLLAMA_MODEL", "qwen2.5:1.5b"), # env var with a default, not a # hardcoded model name — lets
you # swap models (e.g. qwen2.5:3b) # without touching code. "system": system_prompt, # the agent's
role/instructions, kept separate from # the user prompt — mirrors how every real LLM API # (Gemini, OpenAI,
Anthropic) splits system vs user. "prompt": user_prompt, # the actual task input for this specific call.
"stream": False, # get one complete response back, not a token # stream — simpler to log and print for a CLI
demo # that doesn't need live token-by-token output. }, timeout=600, # CPU-only local inference is slow,
especially on longer prompts # 600s, not the requests default (no timeout at all) — long enough that a slow #
synthesis prompt on CPU doesn't get killed mid-generation, but still finite so a # genuinely hung request
doesn't block forever. ) res.raise_for_status() # fail loudly on a non-2xx response (e.g. model not # pulled,
Ollama not running) instead of silently # returning a JSON error body as if it were a reply. return
res.json()["response"] # Ollama's non-streaming response body puts the # generated text under "response" —
pull just that. # 2. SQLite blackboard — shared state every pattern below logs to. In the # "blackboard"
pattern specifically, agents also *read* from it instead of # being handed a value directly, so they're
decoupled from each other. def log_run(agent_name: str, inputs: str, outputs: str): # one write function,
called by every agent # after every call — no agent skips logging. conn = sqlite3.connect(DB_PATH) # open a
fresh connection per call rather than a # long-lived global one — this is a low-frequency, # single-process
script, so connection reuse would # save microseconds at the cost of more state to # manage (e.g. across the
parallel pattern's threads). cursor = conn.cursor() cursor.execute(""" CREATE TABLE IF NOT EXISTS runs ( id
INTEGER PRIMARY KEY AUTOINCREMENT, # simple auto-incrementing id — enough to order # rows and to fetch
"latest" reliably. timestamp DATETIME DEFAULT CURRENT_TIMESTAMP, # DB-assigned timestamp, not Python's #
datetime.now() — keeps clock authority in one # place (the DB) rather than trusting the caller. agent_name
```

```
TEXT, # which agent produced this row — the key every # pattern filters/joins on. input TEXT, # exactly what was sent to the model, logged # verbatim — needed to reconstruct what happened. output TEXT # exactly what the model returned. ) """) # created inline here, not via a schema.sql file — # for one table this small, a separate schema file # is one more file to keep in sync for no real # benefit (see original_shape for the alternative). cursor.execute( "INSERT INTO runs (agent_name, input, output) VALUES (?, ?, ?)", # parameterized query, not an # f-string building SQL — avoids # SQL injection even though the # input here is just LLM text, # not untrusted user input from # a web form; still the correct # default habit. (agent_name, inputs, outputs), ) conn.commit() # persist immediately after every single run — so # a crash mid-pipeline still leaves prior agents' # work durably on disk, not lost with an uncommitted # transaction. conn.close() # close rather than reuse — see note on connect(). def get_latest_output(agent_name: str) -> str: # the "read" side of the blackboard — only used by # run_blackboard(), nothing else calls this. conn = sqlite3.connect(DB_PATH) cursor = conn.cursor() try: cursor.execute( "SELECT output FROM runs WHERE agent_name = ? ORDER BY id DESC LIMIT 1", # newest row for # this agent, not # the oldest — # "latest" means # most recent run, # so a rerun # correctly # supersedes an # earlier one. (agent_name,), ) row = cursor.fetchone() except sqlite3.OperationalError: # table might not exist yet if this is called # before any agent has ever run — caught here # rather than crashing, so the error message below # can be specific instead of a raw traceback. row = None conn.close() if row is None: raise RuntimeError(f"No prior '{agent_name}' entry found on the blackboard.") # fail loudly and # specifically — # not returning an # empty string, # which would # silently feed a # blank context # into the next # agent's prompt. return row[0] # row is a 1-tuple (output,) — unwrap it. # 3. Agent prompts — single source of truth, used by every pattern below. AGENT_PROMPTS = { # one dict, not three separate hardcoded prompt # strings duplicated across every pattern function # — every pattern below looks the agent's name and # prompt up from here instead of repeating them. "extractor": ( "Extractor", # human-readable display name, used in prints and # logged as agent_name in the DB. "You are an expert research agent. Given a technical query, identify the top 3 " "core advancements or breakthrough developments in this field. Output them as a numbered list.", # system prompt asks for exactly 3, numbered — a # constrained output shape is easier for the next # agent (Risk Analyst) to match risk-to-advancement # against, and easier to read on a dashboard card. ), "risk": ( "Risk Analyst", "You are a technical risk analyst. Given a list of advancements in a technology field, " "identify key engineering risks, bottlenecks, or challenges associated with each advancement. " "Respond with a corresponding numbered list matching the advancements.", # explicitly told to match Extractor's numbering — # keeps the two lists aligned 1-to-1 so the # Synthesizer isn't left guessing which risk maps # to which advancement. ), "synthesizer": ( "Synthesizer", "You are a principal technical editor. Synthesize the given advancements and risks " "into a structured executive summary report in Markdown. Highlight key findings, recommendations, " "and a final technical readiness verdict.", # asks for Markdown specifically — dashboard.py # renders this inside a <pre> block, and a # structured report reads better there than an # unstructured paragraph would. ), } # Runs exactly one agent, independently — no other agent's output is involved. # Whatever text is supplied on the command line is fed to that agent directly. def run_single_agent(agent_key: str, query: str) -> str: # returns the output too (not just prints it) — # so run_supervisor() can call this and still # get the value back if it ever needs it. name, system_prompt = AGENT_PROMPTS[agent_key] # unpack straight from the shared dict — this is # exactly why AGENT_PROMPTS exists as a dict keyed # by the same strings argparse's --agent accepts. print(f"=== Running {name} independently ===") # printed banner — this is a live CLI demo, so # stdout narration matters as much as the return # value; the interviewer is watching this scroll by. print(f"[{name}] Processing input: '{query}'...") output = call_llm(system_prompt, query) # query goes straight in as the user prompt, no # reformatting — this agent needs no other agent's # context by definition. log_run(name, query, output) # log immediately after the call, before printing # the result — so a persisted row exists even if # something after this line were to fail. print(f"[{name}] Finished and logged to blackboard.") print(f"\n--- {name} Output ---\n{output}\n") return output # 4a. Pipeline pattern — A -> B -> C, each agent's output is handed directly # to the next as a function argument. Simple, but a bad step early poisons # everything downstream. def run_pipeline(query: str): print(f"=== Pattern: pipeline (sequential) — query: {query} ===") extractor_name, extractor_prompt = AGENT_PROMPTS["extractor"] print(f"[{extractor_name}] Processing query...") advancements = call_llm(extractor_prompt, query) # step 1 — raw query straight in, same as # run_single_agent's extractor call. log_run(extractor_name, query, advancements) print(f"\n--- {extractor_name} Output ---\n{advancements}\n") risk_name, risk_prompt = AGENT_PROMPTS["risk"] risk_input = f"Field: {query}\nAdvancements:\n{advancements}" # step 1's output is interpolated # directly into step 2's prompt string # — this is the "direct handoff" that # defines the pipeline pattern; no DB # round-trip needed to pass it along. print(f"[{risk_name}] Analyzing advancements...") risks = call_llm(risk_prompt, risk_input) log_run(risk_name, risk_input, risks) # log the *combined* prompt as the input, not just # the query — so the DB row shows exactly what this # agent actually saw, not just the original topic. print(f"\n--- {risk_name} Output ---\n{risks}\n") synth_name, synth_prompt = AGENT_PROMPTS["synthesizer"] synth_input = f"Topic: {query}\nAdvancements:\n{advancements}\n\nRisks & Bottlenecks:\n{risks}" # step 3 gets *both* prior outputs — this is why # Synthesizer is last: it's the only agent whose # prompt depends on two upstream results, not one. print(f"[{synth_name}] Synthesizing final report...") report = call_llm(synth_prompt, synth_input) log_run(synth_name, synth_input, report) print(f"\n--- Final Synthesized Report ---\n{report}\n") print("=== Pipeline Complete ===") # 4b. Parallel / independent pattern — all 3 agents run on the same raw query # at the same time. Fast, but no agent can use another's result. def run_parallel(query: str): print(f"=== Pattern: parallel (independent) — query: {query} ===") def run_one(agent_key: str) -> tuple[str, str]: # nested closure, not a module-level function — # it only exists to be the unit of work handed to # the thread pool, so it doesn't need to be visible # or
```

```
reusable outside this one pattern. name, system_prompt = AGENT_PROMPTS[agent_key] print(f"[{name}] Started (independent)...") output = call_llm(system_prompt, query) # every agent gets the *same* raw query — the # defining trait of "independent": nobody's prompt # depends on anybody else's output. log_run(name, query, output) print(f"[{name}] Finished and logged to blackboard.") return name, output # returned as a tuple, not just printed — pool.map # needs a return value to collect results in order. with ThreadPoolExecutor(max_workers=len(AGENT_PROMPTS)) as pool: # exactly 3 workers, one per # agent — no point sizing the pool # larger since there are only ever # 3 units of work to hand out. results = list(pool.map(run_one, AGENT_PROMPTS)) # pool.map over the dict iterates its keys # ("extractor", "risk", "synthesizer") — chosen # over submit()+as_completed() because we don't # need first-finished-first-handled ordering, # just "all three, then show results in a # stable order." for name, output in results: print(f"\n--- {name} Output ---\n{output}\n") # printed after the pool block closes — so the # per-agent "Started"/"Finished" lines interleave # live as threads run, but the final full outputs # print in one clean, deterministic block after. print("=== Parallel Run Complete ===") # 4c. Orchestrator-worker / supervisor pattern — a central controller decides # which agent(s) to run, in what order, given the query. Here the controller # is a lightweight router call: it decides whether the query is a simple # lookup the Extractor can fully answer, or whether it needs the full chain. SUPERVISOR_PROMPT = ( # module-level constant, not built fresh inside the # function — it never changes per call, so there's # no reason to reconstruct the string every run. "You are a routing controller in front of a 3-agent system:\n" "- Extractor: identifies the top advancements/facts for a topic.\n" "- Risk Analyst: analyzes engineering risks — needs Extractor's output first.\n" "- Synthesizer: writes a final executive report — needs both prior outputs.\n" "Given the user's query, decide: does it need the Risk Analyst and Synthesizer too " "(a request for risks, bottlenecks, or a full report/analysis), or is it a simple " "factual/lookup question the Extractor alone can fully answer?\n" "Respond with exactly one word, nothing else: 'extractor' or 'pipeline'." # forced to exactly one word — a small local model # is unreliable at following complex output-format # instructions, so the classification is kept to # the simplest possible parseable output: one word. ) def run_supervisor(query: str): print(f"=== Pattern: supervisor (orchestrator-worker) — query: {query} ===") decision = call_llm(SUPERVISOR_PROMPT, query).strip().lower() # .strip().lower() to normalize — # small local models often pad # responses with whitespace or # inconsistent casing even when told # to answer in one word. decision = "extractor" if "extractor" in decision and "pipeline" not in decision else "pipeline" # substring check, not exact `== "extractor"` — # more forgiving of a model that answers # "Extractor." or "extractor\n" instead of the bare # word; and defaults to "pipeline" (the safer, more # thorough option) on any ambiguous or unparseable # response, rather than silently under-running. print(f"[Supervisor] Classified query as '{decision}'.") # decision is printed before acting on it — # so the routing choice is visible and # explainable live, not a silent branch. if decision == "extractor": run_single_agent("extractor", query) # reuses run_single_agent rather than duplicating # its logic — the supervisor's job is only to # decide, not to reimplement how a single agent runs. else: run_pipeline(query) # reuses run_pipeline rather than duplicating it — # same reasoning: the supervisor composes the other # patterns, it doesn't reinvent them. # 4d. Blackboard pattern — agents never call each other directly. Each agent # reads whatever it needs from the shared blackboard (the SQLite table), # rather than being handed a value by the caller. Same execution order as # pipeline, but fully decoupled: swap or rerun any agent without touching # the others, since they only ever talk through shared state. def run_blackboard(query: str): print(f"=== Pattern: blackboard — query: {query} ===") extractor_name, extractor_prompt = AGENT_PROMPTS["extractor"] print(f"[{extractor_name}] Processing query...") advancements = call_llm(extractor_prompt, query) log_run(extractor_name, query, advancements) # write to the blackboard — this row is what the # next agent will read back, not receive directly. print(f"[{extractor_name}] Wrote result to blackboard.") risk_name, risk_prompt = AGENT_PROMPTS["risk"] print(f"[{risk_name}] Reading '{extractor_name}' result from blackboard...") advancements_from_board = get_latest_output(extractor_name) # deliberately re-fetched from the DB # instead of reusing the `advancements` # variable already sitting in memory — # that in-memory reuse is exactly what # pipeline does; blackboard's whole # point is that this agent only trusts # the shared table, never a value handed # to it directly by the caller. risk_input = f"Field: {query}\nAdvancements:\n{advancements_from_board}" risks = call_llm(risk_prompt, risk_input) log_run(risk_name, risk_input, risks) print(f"[{risk_name}] Wrote result to blackboard.") synth_name, synth_prompt = AGENT_PROMPTS["synthesizer"] print(f"[{synth_name}] Reading prior results from blackboard...") advancements_from_board = get_latest_output(extractor_name) # fetched *again* here, independently of # the Risk Analyst's fetch above — this # agent doesn't trust a variable handed # down through the function either; it # does its own read, same as every other # agent in this pattern. risks_from_board = get_latest_output(risk_name) synth_input = ( f"Topic: {query}\nAdvancements:\n{advancements_from_board}" f"\n\nRisks & Bottlenecks:\n{risks_from_board}" ) report = call_llm(synth_prompt, synth_input) log_run(synth_name, synth_input, report) print(f"[{synth_name}] Wrote result to blackboard.") print(f"\n--- Final Synthesized Report ---\n{report}\n") print("=== Blackboard Run Complete ===") PATTERNS = { # maps the --pattern CLI string straight to the # function that implements it — chosen over an # if/elif chain in __main__ so adding a fifth # pattern later means adding one dict entry, not # another branch in the argument-handling code. "pipeline": run_pipeline, "parallel": run_parallel, "supervisor": run_supervisor, "blackboard": run_blackboard, } if __name__ == "__main__": # guards the CLI entry point — lets this file be # imported (e.g. by dashboard.py or a test) without # triggering a live LLM run as a side effect. parser = argparse.ArgumentParser( description="3-agent demo: choose a multi-agent pattern, or run any one agent independently." ) parser.add_argument( "query", nargs="?", default="how much does a pen costs", # positional and optional — so the # script is runnable with zero # arguments for a quick smoke test, # but takes a real topic when given one. help="Task/topic text to feed the agent(s)", )
```

```
parser.add_argument( "--agent", choices=list(AGENT_PROMPTS), default=None, # choices=... restricts input to the three # valid keys — argparse rejects a typo'd # agent name before any LLM call is made, # instead of failing deep inside call_llm. help="Run only this agent, independently — overrides --pattern", ) parser.add_argument( "--pattern", choices=list(PATTERNS), default="supervisor", # defaults to supervisor, not # pipeline — supervisor is the one # pattern that itself decides # independent-vs-collaborative per # query, which is the literal # requirement in the brief; the # other three are fixed shapes you # opt into deliberately. help="Multi-agent pattern to run (default: supervisor)", ) args = parser.parse_args() if args.agent: run_single_agent(args.agent, args.query) # --agent short-circuits pattern selection # entirely — if you know exactly which one agent # you want, there's no reason to run the supervisor # classification call first. else: PATTERNS[args.pattern](args.query) # dict lookup + call — same reasoning as PATTERNS # itself: one line, no branching to maintain.
```

`requirements.txt`

```
python-dotenv>=1.0.0 # loads .env into os.environ — only real alternative was asking the user to set # OLLAMA_MODEL / etc. as real shell env vars every session, which is worse UX for a # rehearsal script you'll run many times. requests>=2.30.0 # synchronous HTTP client for the one POST call to Ollama — chosen over the stdlib's # urllib (more boilerplate for the same result) or httpx (adds async support this # script never uses, since call_llm is a simple blocking call by design).
```

Notice `google-genai` is **not** in this file — it was removed on purpose when `leaner_cut` was cut over to Ollama-only. If you re-add a real Gemini path later, that's the dependency that comes back.

---

7. Pattern comparison table

Dimension	Pipeline	Parallel	Supervisor	Blackboard
Execution order	Sequential: A → B → C, one after another	Concurrent: A, B, C all fire at once (threads)	Decided at runtime: either just A, or the full A→B→C chain	Sequential: A → B → C, same order as pipeline
How data moves between agents	Direct handoff — prior output passed as a Python function argument	No handoff — each agent gets the same raw query, independently	Same as whichever it picks (single-agent = no handoff, pipeline = direct handoff)	Indirect — each agent writes to SQLite, the next agent re-reads it from SQLite, ignoring any in-memory value
Coupling between agents	Tightly coupled — B's call requires A's return value in scope	None — agents don't know about each other at all	Coupled only for the chain it picks; decoupled if it picks single-agent	Loosely coupled — agents only share a DB table, never call/reference each other directly
Who decides what runs	Fixed at code level — always all 3, in order	Fixed at code level — always all 3, no order	An LLM call (the router) decides live, based on the query	Fixed at code level — always all 3, in order
LLM calls made	Always 3	Always 3	1 (simple query) or 4 (router call + 3 agents)	Always 3
Can one agent be rerun alone, safely?	No — needs the caller to still have the prior variable	Trivially — no dependency exists in the first place	Depends which branch it took	Yes — it just re-reads the DB, doesn't need the original process/variables alive
Survives a crash between steps?	No — in-memory `advancements`/`risks` are lost, must restart from step 1	N/A — no sequencing to resume	Same as whichever branch it's mid-way through	Yes — whatever was written to the DB before the crash is still there for the next agent to read

Matches the JD's "independent OR collaborative depending on task"?	Only demonstrates collaborative	Only demonstrates independent	Demonstrates both — decides per query, which is the literal requirement	Only demonstrates collaborative (different mechanism, same behavior)
When you'd actually pick it	Simple, auditable, everything genuinely depends on the last step	Truly unrelated sub-tasks on the same input, want speed	You don't know in advance which shape a given query needs	Agents might run in separate processes/restarts, or you want to swap/rerun one agent without touching the others

The one-line version: **pipeline** and **blackboard** produce the same output in the same order — they only differ in *mechanism* (direct handoff vs. shared-state read). **Parallel** is the only one that's actually independent. **Supervisor** is the only one that *decides* between shapes rather than being a fixed shape itself.

---

8. Likely interviewer questions about this file — with answers

"Why did you build one file instead of separate modules or classes?"

> I keep it in one file because there are only three agents and one shared LLM caller — splitting that into multiple files or a class hierarchy adds import overhead and indirection without buying me anything, since nothing here is reused outside this script. I'd reach for classes or separate modules the moment I have real polymorphism to express or need to unit-test pieces in isolation.

"Why SQLite instead of Postgres or another database?"

> I need something that persists rows and needs zero setup — SQLite gives me both, as a single file with no server process to run or configure. If this needed concurrent writers at scale, or ran across multiple machines, I'd move to Postgres — but for one process logging its own runs, that's unnecessary infrastructure for a demo.

"Why does the table get created inline instead of using a schema.sql file?"

> There's exactly one table, with one shape, that never changes. A separate schema file is one more file to keep in sync for zero real benefit at this scale — `original_shape` uses a schema file instead, so I can show both approaches and explain when each earns its cost.

"Why Ollama instead of a real API like Gemini or OpenAI?"

> Local inference means unlimited calls with no quota and no network dependency while I'm rehearsing or iterating — I'm not burning a rate-limited free tier every time I test a change. The `call_llm` function is the only place that knows how to reach the model, so swapping in a real provider later is a one-function change, not a rewrite.

"Walk me through what happens when I run this with no arguments at all."

> `query` defaults to a sample string, and `--pattern` defaults to `supervisor` — so with zero arguments, the supervisor pattern runs a quick classification call on that sample query, decides whether it needs just the Extractor or the full chain, and runs whichever it picked.

"What's the actual difference between your pipeline and blackboard patterns? They look identical."

> They execute in the same order and produce the same shape of output, but the mechanism for passing data between agents is different. Pipeline hands the prior agent's return value directly into the next function call — pure Python. Blackboard writes every result to SQLite, and the next agent deliberately ignores anything in memory and re-reads its input from the database. The value only shows up once you break the "everything happens start-to-finish in one process" assumption — a crash mid-run, or rerunning one agent alone, works cleanly in

> blackboard and doesn't in pipeline.

"If pipeline and blackboard behave the same in this demo, why implement both?"

> Because the interviewer or the task might specifically ask for agents that don't call each other directly, and I want the actual mechanism ready, not just the same output produced a different way in my head. It's also a stronger answer than claiming I understand the trade-off — I can point at the exact two lines that differ.

"How does the parallel pattern actually achieve concurrency?"

> I run all three agent calls inside a `ThreadPoolExecutor` with one worker per agent, so three HTTP requests to Ollama are genuinely in flight at the same time rather than issued one after another. I'd be honest that the local Ollama server may still process them one at a time under the hood on a single GPU — the concurrency is real at my code's level; whether the backend parallelizes it further is a separate, honest caveat.

"Why threads and not asyncio?"

> Every LLM call here is a blocking `requests.post` — asyncio would need the whole call chain rewritten around an async HTTP client to actually benefit. Threads parallelize blocking I/O with the code already written exactly as it is, which is the simpler change for three calls.

"What does the supervisor pattern actually add over just always running the full pipeline?"

> It stops the system from burning three LLM calls on a query a single agent could already answer completely. Concretely, it makes one cheap classification call first — does this need risk analysis and synthesis, or is it a simple lookup — and only then runs whatever that decision actually requires. That's also the one pattern that literally satisfies "agents work independently or collaboratively depending on the task," since it decides that per query instead of me deciding it in advance with a flag.

"How reliable is that classification? What if the model gets it wrong?"

> Honestly, not perfectly reliable on a 1.5B local model — that's why I default the fallback to `"pipeline"`, the more thorough option, rather than `"extractor"`, if the response is ambiguous or doesn't parse cleanly. Under-running is a worse failure than occasionally running one extra agent. There's also `--pattern pipeline` explicitly, if I need to guarantee the full chain regardless of what the router decides.

"What happens if I run just the Synthesizer alone with `--agent synthesizer`?"

> Right now it takes whatever text you typed as the direct prompt, not the real prior outputs from the blackboard — so if you haven't given it actual advancements-and-risks text, it has nothing concrete to synthesize and falls back to explaining the concepts abstractly. That's a real, known gap in the current `--agent` mode: it doesn't yet pull prior context from the DB the way the blackboard pattern does. I'd fix that by making `--agent risk` and `--agent synthesizer` automatically read the latest matching row from SQLite when run standalone.

"Why does `run_blackboard` re-fetch the Extractor's output from the DB twice — once for Risk Analyst, once for Synthesizer — instead of reusing the first fetch?"

> That's deliberate, not an oversight. The whole point of the blackboard pattern is that every agent reads fresh from shared state rather than trusting a value some other part of the code already fetched — if I reused the first fetch, I'd quietly be back to direct in-memory handoff, just with extra steps.

"Why is `AGENT_PROMPTS` a dictionary instead of three separate functions with hardcoded prompts?"

> Every pattern needs to look up an agent's display name and system prompt by key — putting that in one dictionary means I tune a prompt in exactly one place, and every pattern, plus `--agent` mode, picks the change up automatically. Three separate hardcoded prompt strings would drift out of sync the moment I changed one and forgot the other two.

"How do you handle a failure — say, Ollama isn't running, or the model isn't pulled?"

> `res.raise_for_status()` right after the request means any non-2xx response — model not found, server down — raises immediately with a real HTTP error, instead of silently treating an error body as if it were a valid response. I'd rather see a loud, specific traceback live than debug a nonsense answer that came from swallowed failure.

"Why not add retries for a failed Ollama call, the way `original_shape` retries Gemini on rate limits?"

> Rate-limit retry exists for Gemini specifically because 429s are an expected, transient condition on a free tier with a documented wait time. A local Ollama failure — server down, model missing — isn't transient in the same way; retrying blindly wouldn't fix a model that was never pulled. The right response there is to fail loudly and tell you exactly what's wrong, not mask it with a retry loop.

"How do you know the dashboard is actually showing the latest data?"

> It isn't live — `dashboard.py` does one query against `runs` each time it's run and writes a static HTML file. If I run more agents after generating it, the page won't update until I run `dashboard.py` again; the "Refresh" link on the page just reloads the same static file, it doesn't requery the database. I'd upgrade to Streamlit with an actual requery-on-refresh if a live view were a real requirement, not a guess.

"Why print so much to stdout instead of just returning values and logging quietly?"

> This is meant to be watched live during a demo — the interviewer needs to see what's happening as it happens, not just a final return value. Every agent announces when it starts, when it finishes, and what it produced, so the reasoning is visible in real time, not just in the database afterward.

"What's the actual trade-off of choosing SQLite over an in-memory Python dict for the blackboard?"

> An in-memory dict dies with the process — if I want any agent to survive a restart, or want to inspect results after the run without keeping the process alive, I need something durable. SQLite gives me that durability essentially for free, as a single file, with no server to manage.

"Could you scale this to more than three agents?"

> The pattern functions and `AGENT_PROMPTS` dict both scale by adding entries, not by restructuring — a fourth agent is one more dictionary entry and one more line in whichever pattern needs it. Where it would actually start to strain is the supervisor's routing prompt — a one-word "extractor vs pipeline" decision doesn't generalize cleanly past a handful of agents; past that I'd want a more structured decision (e.g. the model naming which agent keys to call, parsed as JSON) rather than a single hardcoded word.

"Why didn't you use LangChain, LangGraph, or CrewAI for this?"

> For three agents and four wiring patterns, the actual orchestration logic is maybe 100 lines, and I can explain every one of those lines live. A framework buys shared vocabulary and tooling once you're past a real scale — many agents, complex conditional routing, multiple people maintaining it — but here it would add abstraction I'd have to explain the internals of anyway, for no functional gain.

"Where would you put a human-in-the-loop checkpoint in this specific system, if asked to add one?"

> Right before `log_run` commits the Synthesizer's final report, since that's the irreversible, externally-visible output — an approval gate there costs nothing on the common path and catches the one output that actually matters if something upstream went wrong. I wouldn't gate Extractor or Risk Analyst individually; their outputs are intermediate and cheap to regenerate.

"How would you productionize this beyond a local demo?"

> Swap Ollama for a hosted model behind `call_llm` — that's the one function that needs to change. Move `blackboard.db` to a networked Postgres instance for concurrent access. Replace the static HTML dashboard with a small Streamlit or FastAPI view that queries live. Everything else — the agent prompts, the four patterns, the logging shape — stays the same, because none of that logic is tied to running locally.

"What would you have done differently with more time?"

> I'd make `--agent` mode blackboard-aware for Risk Analyst and Synthesizer, so running them standalone pulls real prior context instead of requiring the full pattern to have already run. I'd also make the supervisor's routing decision structured (JSON with an explicit agent list) instead of parsing a single free-text word, since that's the most fragile part of the whole file on a small local model.

"Why is `query` a positional argument with `nargs="?"` instead of just a required argument or a `--query` flag?"

> I wanted this runnable with zero arguments for a quick smoke test — `nargs="?"` with a default means `python main.py` alone still does something sensible, instead of immediately erroring with "missing required argument." A `--query` flag would work too, but the topic is the one thing you always supply, every single run — making it positional means less typing for the thing you type most.

"Why does `--agent` use `choices=list(AGENT_PROMPTS)` instead of just `choices=["extractor", "risk", "synthesizer"]`?"

> Same reason `AGENT_PROMPTS` is a dictionary in the first place — one source of truth. If I ever rename a key or add a fourth agent, the CLI's valid choices update automatically because they're derived from the dict, not typed out separately somewhere that could drift out of sync.

"Why load `.env` with an explicit path instead of just calling `load_dotenv()`?"

> `load_dotenv()` with no arguments searches upward from the current working directory, not from wherever this script physically lives — so it silently finds nothing if you run this from the repo root instead of from inside `leaner_cut`. Pointing it at `Path(__file__).parent / ".env"` means it always finds the right file regardless of which directory you launched Python from.

"You have `original_shape` doing the same three-agent job in a different style — why maintain two versions?"

> They answer two different questions. `original_shape` is the OOP version — a `BaseAgent` class, subclasses per role, a `Database` class wrapping SQLite, a separate `schema.sql`, and a Streamlit dashboard. `leaner_cut` is the same three agents as plain functions in one file, no classes, a static HTML dashboard. Having both means I can speak to either style depending on what the interviewer's actually testing — "can you structure this with proper OOP" versus "can you get something small working fast" are different questions, and I didn't want to be caught only knowing one answer to "why did you structure it this way."

"If pipeline and blackboard produce identical output, how would a grader watching over your shoulder even know you used blackboard instead of pipeline?"

> Honestly, from the printed output alone, they mostly couldn't — both print the same "Processing... Wrote/logged... Output" sequence in the same order. The tell is in the code, not the transcript: blackboard's Risk Analyst and Synthesizer steps call `get_latest_output()` instead of referencing the `advancements`/`risks` variables already in scope. That's exactly why I'd never just describe blackboard verbally and leave it there — I'd point at those two specific lines and explain why they deliberately don't take the shortcut sitting right next to them.

"What's the worst input you could give this system that would actually break it?"

> An empty string or pure whitespace as the query wouldn't crash anything — it'd just produce a vague or empty-ish response, since nothing here validates that the query is meaningful before sending it to the model. What would actually break it is a query that happens to make the supervisor's classifier respond with neither "extractor" nor "pipeline" in any recognizable form — say a model hiccup that returns something totally malformed. That's exactly

> why the fallback in `run_supervisor` defaults to `"pipeline"` on anything ambiguous, rather than crashing or defaulting to running nothing.

"What happens if Ollama itself hangs or times out mid-demo?"

> `call_llm` sets `timeout=600` on the request, so after ten minutes it would raise a `requests.exceptions.Timeout` rather than hang forever — but in a live 1-hour session, even 30 seconds of silence looks bad. If I hit that live, I'd say out loud what's happening — "the local model's taking longer than expected, let's check `ollama ps`" — rather than sit there quietly waiting, since narrating a slow dependency is still narrating my debugging process, which is the thing actually being scored.

"Could two agents ever write to the database at the exact same moment and collide?"

> Not in the pipeline, supervisor, or blackboard patterns — those are strictly sequential, one `log_run` call finishes before the next agent starts. Parallel is the one place it's theoretically possible, since three threads call `log_run` around the same time — but each call opens its own short-lived SQLite connection, does one commit, and closes, and SQLite serializes writes at the file level by default. So worst case a write briefly waits on another; it doesn't corrupt data. If this were high-throughput instead of three calls, I'd want a connection pool or a single writer thread instead of relying on SQLite's default locking.

"Why does `requirements.txt` only have two lines? Isn't that too thin for an LLM project?"

> It's thin because the two things this file actually needs are an HTTP client (`requests`) and a `.env` loader (`python-dotenv`) — everything else is Python's standard library (`sqlite3`, `argparse`, `concurrent.futures`, `pathlib`). There's no `google-genai` in here on purpose — that dependency came out when I cut `leaner_cut` over to Ollama-only, since talking to Ollama is just a raw HTTP POST, no SDK required.

"If you had to explain `call_llm` to someone who's never seen an LLM API before, in ten seconds, what would you say?"

> It's a function that sends two pieces of text — instructions for the model, and the actual question — to a model running on this machine, and gets back whatever text the model generated in response. Everything else in this file is just deciding which instructions to send, and what to do with the text that comes back.

---

9. Pending / known gaps (not yet fixed)

Things surfaced during rehearsal that are still open, called out explicitly so they don't get lost:

- **`--agent risk` / `--agent synthesizer` don't read prior context from the blackboard.** They take your CLI text as the direct prompt, so standalone runs of these two only work well if you paste real advancements/risks text yourself — otherwise the model answers abstractly (see the Q&A above).
- **DB Browser for SQLite (or any external tool with `blackboard.db` open) will lock the file** and cause `sqlite3.OperationalError: database is locked` on the next `main.py` run — close it before running agents.
- **`main copy.py`** exists alongside `main.py` in this folder — worth deleting or reconciling if it's stale, so it doesn't get confused for the live file during a screen-share.
- **`run_rehearsal.ps1` doesn't know about `--pattern`** — it only wires up `--agent` vs. the full pipeline; `parallel`/`blackboard`/`supervisor` need a direct `main.py` call (see §5).

---

10. Broader interview-prep reference

The material below is merged from `agentic_ai_interview_guide.md` — it goes beyond this one file into the wider taxonomy, JD, and prep context around the live-build round. Kept here so there's one document instead of two.

10.1 Decoding the job description

Stripped to what's actually being tested:

JD line	What they're really checking
"Universal Worker Model (UWM) architecture"	**No verified public reference for "UWM" as a named, standard agentic architecture** — very likely this company's internal name for their agent framework. Don't bluff a definition — ask directly: *"Can you point me to your internal docs on UWM, or describe its core primitives?"* That question itself signals seniority.
"Fully autonomous and semi-autonomous agents... multi-step task planning"	Loop-based agents (§10.4) vs. single-shot agents — know when each applies
"Agent-to-Agent (A2A) communication"	Multi-agent coordination patterns (§7 above) — specifically whether you can name more than one
"RAG pipelines and knowledge-driven agent architectures"	Vector DB + retrieval + grounding basics (§10.7)
"Memory management, context handling, agent lifecycle"	Short-term (context window) vs. long-term (vector store / DB) memory distinction
"Human-in-the-loop (HITL)"	Can you describe an approval/interrupt point in an agent loop, not just automate everything
"PostgreSQL... stored procedures, indexing, query optimization"	Straight DB skill
"AWS: EC2, Lambda, ECS/EKS, S3, RDS, API Gateway, CloudWatch, IAM"	Standard cloud deployment vocabulary (§10.8)
"OAuth2, JWT, API security"	Auth basics for agent-exposed endpoints
"LangChain / LangGraph (preferred), CrewAI / AutoGen / OpenAI Agents SDK"	Know what these are and how they differ, not necessarily be deep in all of them (§10.6)
"MCP and emerging agentic standards"	Model Context Protocol — know the one-sentence version
"HIPAA / data governance"	Only relevant if the healthcare-domain work comes up

Read as an interviewer: this JD is written broad. The 2nd-round live-build is a *practical filter* — can you actually construct a working multi-agent system under time pressure, not just talk about one. The rest of the JD's surface (AWS, RAG, security, frameworks) is very likely probed in a separate technical/architecture conversation.

10.2 Decoding the 2nd-round problem statement

- **1 hour, live, screen-share.** Scored on *process*, not polish.
- **Three small, lightweight agents.** "Lightweight" is doing real work in that sentence — it's telling you not to over-engineer.
- **Independent or collaborative, decided per task.** The task is revealed on the day — you can't pre-wire the collaboration pattern, only pre-build the *capability* to wire it either way.
- **Gather/process info via LLM and/or API calls.** Not every agent needs to call an LLM — one could just be a plain API-calling function. Don't assume all three must hit an LLM.
- **Database persists outputs and execution logs.** Two things, one table is fine: each row is both an output record and a log line.

- **Dashboard displays results.** No spec on *how* — a printed table, a static HTML file, or a running web app are all valid; the ask is "displays," not "is a web app."
- **AI tool use is allowed, blind copy-paste is not.** You must be able to explain every line, live.

10.3 What actually is an "agent"?

There's no single universally-agreed definition. The most useful working definition for this interview:

> **An agent is a system that observes some state, decides on an action using an LLM and/or rules, takes that action (which may include calling tools/APIs), and optionally repeats.**

The critical distinguishing feature vs. a plain script: **the decision of *what to do next* is at least partly made by the model, not hard-coded entirely in advance.** A single LLM call that returns an answer is *not* usually called an agent — it's a completion. The word "agent" earns itself when there's a **decision + action** step, even if that loop only runs once (single-shot).

10.4 Taxonomy: types of agents

Classical AI taxonomy (pre-LLM, standard university-course material):

Type	How it decides	Example
Simple reflex agent	Fixed rule triggered by current perception only, no memory	Thermostat
Model-based reflex agent	Keeps an internal model of the world to handle partial observability	Robot vacuum tracking a map
Goal-based agent	Chooses actions that lead toward an explicit goal state	A* pathfinding agent
Utility-based agent	Chooses actions that maximize a utility/value function, not just "any path to goal"	An agent trading off speed vs. cost
Learning agent	Improves its decision policy over time from feedback	RL agent

Modern LLM-agent taxonomy (practitioner convention, not a single formal standard):

Type	Shape	When to use
Single-shot / tool-calling agent	One `input → LLM (with tool access) → output`, no re-planning loop	Task is well-scoped, one round of reasoning is enough — **this is what all three of `leaner_cut`'s agents are**
ReAct agent (Reason + Act, interleaved)	Loop: think → pick a tool → observe result → think again → ... → answer	Task needs iterative lookup where the number of steps isn't known upfront
Plan-and-execute agent	Produces a full plan upfront, then executes each step (optionally re-planning on failure)	Long multi-step tasks where planning cost is worth paying once
Reflexion / self-critique agent	Generates an output, critiques its own output, revises	Tasks where quality matters more than speed and there's a way to self-verify
Multi-agent system	Multiple single-shot or looped agents coordinating	Task naturally decomposes into specialist roles — this is `leaner_cut` itself

Interview-ready one-liner if asked "how many types of agents are there": *"There isn't one official count — classical AI names five reflex/goal/utility/learning-style categories, and the LLM-agent world commonly talks about single-shot, ReAct, plan-and-execute, reflexion, and multi-agent — but that second list is convention, not a formal standard, and people draw the lines slightly differently."*

10.5 Core building blocks of any agent system

Regardless of framework or no-framework, every agent system is assembled from the same six pieces:

1. **LLM/API call wrapper** — one function, one place to change providers/models (`call_llm` in this file)
2. **Tool/action interface** — how the agent's decision turns into a real action (API call, DB write, function call)
3. **State/memory** — what the agent knows: short-term (this conversation/context) vs. long-term (DB, vector store)
4. **Control flow** — single-shot vs. loop vs. plan-and-execute
5. **Stop condition** — max iterations, a model-emitted "done" signal, or a budget check — **mandatory**, not optional, for anything that loops
6. **Persistence + observability** — logging every input/output so failures are visible, not silent (`log_run` in this file)

10.6 Framework landscape

Framework	One-line description	Fit
LangChain	General-purpose LLM app framework: chains, tool-calling, retrieval components	Broad tooling, sometimes more abstraction than a small agent needs
LangGraph	Graph-based orchestration for agents, built by the LangChain team, models agent flow as nodes/edges with explicit state	Closest fit to the JD's "supervisor"/A2A language
CrewAI	Higher-level abstraction: define agents by "role," let the framework handle delegation	Fast to prototype role-based multi-agent setups
AutoGen (Microsoft)	Multi-agent conversation framework — agents literally "talk" to each other in a loop	Closest fit to a true peer-to-peer/A2A pattern
OpenAI Agents SDK	OpenAI's own lightweight agent/tool-calling SDK	Newer; verify current maturity/API before claiming depth
MCP (Model Context Protocol)	An open protocol standardizing how an LLM application connects to external tools/data sources	Know this one-sentence definition; it's explicitly named in the JD

10.7 RAG & vector DB basics

RAG in one sentence: instead of relying only on what's in the model's training data, you retrieve relevant text chunks from an external knowledge store at query time and inject them into the prompt, so the model reasons over grounded, current information.

Minimal pipeline shape: **chunk** source documents → **embed** each chunk into a vector → **store** vectors in a vector database → **retrieve** the top-k most similar chunks to the query at runtime → **inject** retrieved chunks into the LLM prompt as context → **generate** the grounded answer.

Common vector DB options named in the JD: **Pinecone, Weaviate, Qdrant, pgvector** (a Postgres extension — could keep everything in one Postgres instance via pgvector instead of adding a separate vector DB service).

10.8 AWS deployment concerns for agent systems

AWS service	Role in an agent system
EC2 / ECS / EKS	Where the orchestrator/agent processes actually run
Lambda	Good fit for single-shot, stateless agent invocations triggered by events
RDS (Postgres)	Production equivalent of this file's SQLite blackboard — same table concept, networked and concurrent-safe
S3	Document/artifact storage — e.g., source documents for a RAG pipeline
API Gateway	Exposes agent endpoints to external callers with auth/throttling
CloudWatch	Logs and metrics — production equivalent of the `runs` table's logging role, but for infrastructure-level observability
IAM	Least-privilege access control between services

If asked "how would you productionize this," a credible answer chains these: agents on ECS/Lambda, blackboard becomes RDS Postgres, logs also go to CloudWatch, S3 for any document store feeding RAG, API Gateway + IAM in front of anything externally callable.

10.9 Security, RBAC, and enterprise concerns

- **OAuth2 / JWT**: standard token-based auth for API access — JWT is a signed token carrying claims (identity, scope) that a service verifies without a DB round-trip; OAuth2 is the broader authorization framework token issuance sits inside.
- **RBAC (Role-Based Access Control)**: permissions attached to roles, not individual users. Attribute-level RBAC is a step beyond basic RBAC — it checks specific attributes/conditions, not just a role label.
- **HIPAA**: US healthcare data privacy regulation — implies encryption at rest/in transit, access logging, and minimal necessary data exposure. Detailed compliance work is normally deferred to legal/compliance while handling the technical controls.

10.10 Handling failure live — the recovery playbook

The loop for any live failure: 1. **Name the failure class out loud** — malformed output, hallucinated API/method, traceback, or under-specified prompt. 2. **Read the actual error, don't guess** — for a traceback, find the exact failing line before touching code. 3. **Make one targeted fix** — never regenerate the whole file blind. 4. **State why you expect this fix to work** before re-running. 5. **If it fails again, that's new information, not a reason to panic** — narrate the update to your hypothesis.

Never do these live: silently retry, silently swallow an exception, regenerate a whole file hoping it's different, defend a hallucinated detail because "the AI said so."

10.11 Best & simplest practices (what a senior engineer keeps vs. cuts)

Situation	Junior instinct	Senior instinct	Why
Need 3 agents	Build a `BaseAgent` class hierarchy	Plain functions, unless you're actually swapping implementations	Classes earn their cost with reuse/testing over time — a one-shot demo has neither

Need to loop	Add a ReAct loop "to be safe"	Default to single-shot; add a loop only to the one agent whose task is open-ended	Unearned complexity is exactly what a "why did you do this" question will expose
Need a DB	Reach for Postgres/Mongo	SQLite, unless concurrency or scale is a real requirement	Zero setup, one file, inspectable in one line, live
Need a dashboard	Spin up a server	Static HTML from a DB query, unless live-refresh is explicitly required	No port/network failure mode
LLM output malformed	Silently retry	Say what broke, tighten the prompt/schema, retry with a stated reason	Narrated debugging is the thing being scored, not the happy path
Framework available	Reach for LangChain/CrewAI by default	Hand-roll for small agent counts under time pressure; frameworks earn their cost at 10+ agents with complex routing	You can explain every line of your own code; you can't always explain library internals live
Asked "why not X"	Defend the choice you made	State the trade-off honestly, including what you'd change with more time	Shows judgment, not attachment

The single unifying principle: **every piece of your system should map to a literal line in the problem statement.** If you can't point to which requirement a piece of code satisfies, it's ceremony, and ceremony is what gets cut when someone asks "why is this here."

10.12 More likely interview questions — beyond the JD's live-build round

"What's the difference between RAG and fine-tuning, and when would you choose one over the other?"

> RAG injects external knowledge at query time without changing model weights — it's faster to update (just change the source documents) and keeps the model's reasoning general. Fine-tuning bakes behavior or domain knowledge into the weights themselves — better for changing *style* or *task format*, not for keeping facts current. If the underlying facts change often, I'd reach for RAG first.

"How would you handle memory across a long-running agent conversation?"

> Two tiers: short-term is just what fits in the context window for the current turn. Long-term is anything that needs to survive across sessions — I'd persist that outside the model, in a DB or vector store, and retrieve only the relevant slice back into context per turn, rather than trying to keep everything in-context indefinitely.

"What is Model Context Protocol, and why does it matter?"

> It's a protocol that standardizes how an LLM application connects to external tools and data sources, so you write one integration against the protocol instead of a bespoke one per tool per vendor.

"Where would you put a human-in-the-loop checkpoint in an agent pipeline, and why there?"

> Wherever an action is irreversible or high-cost if wrong — e.g., before an agent sends an external communication, commits a financial transaction, or modifies production data. Cheap, reversible, or purely informational steps don't need it; adding approval gates everywhere just kills the automation value.

"How do you decide between a hand-rolled agent and using LangGraph/CrewAI/AutoGen in production, not in an interview?"

> Scale and team size, mainly. A few agents with simple routing — hand-rolled is fine and gives full visibility. Once you're past roughly 10+ agents with complex conditional routing, retries, and multiple people maintaining it, a

> framework's shared vocabulary and tooling starts paying for its abstraction cost.

10.13 A realistic 1-hour rehearsal timeline

Time	What you're doing
0–5 min	Task is revealed. Say out loud: what are the 3 agents, are they collaborative or independent, what does each one's input/output look like
5–15 min	Write `call_llm`, `log_run`, and one agent function. Run it once against a throwaway input to prove the wiring works before building the other two
15–30 min	Write agents 2 and 3, wire the orchestrator per the decided mode
30–40 min	Run the full pipeline end to end. Expect at least one bug — narrate it per §10.10, don't panic
40–50 min	Write and run the dashboard script, confirm it reads real rows from the DB
50–60 min	Open the DB directly in a terminal to show real persisted rows; answer questions; state what you'd add with more time

10.14 Final pre-interview checklist

- [ ] `call_llm` tested against a real key/model, today, not assumed to still work from last week
- [ ] Confirm current model name/free-tier limits for whichever provider you're using
- [ ] One full pipeline run completed today, not just "it worked before"
- [ ] Can state, without looking, why single-shot over a loop for this task shape
- [ ] Can name at least two multi-agent coordination patterns beyond the one you built
- [ ] Can name at least one classical-AI agent type and one LLM-agent type without conflating the two taxonomies
- [ ] Comfortable saying "I'd need to verify the current API for that" out loud, rather than guessing at syntax under pressure
- [ ] Have an honest, rehearsed answer for "what would you have done differently with more time"

10.15 Things to verify yourself before the interview

- Current model names and free-tier limits for whichever LLM provider you commit to (changes frequently)
- Current SDK method names/call shape for that provider
- What "Universal Worker Model" specifically means at this company — ask them, don't guess
- Current LangGraph/CrewAI/AutoGen/OpenAI Agents SDK capabilities if the conversation goes deep on frameworks
- Current MCP spec details if asked to go beyond the one-sentence definition